Integration Testing

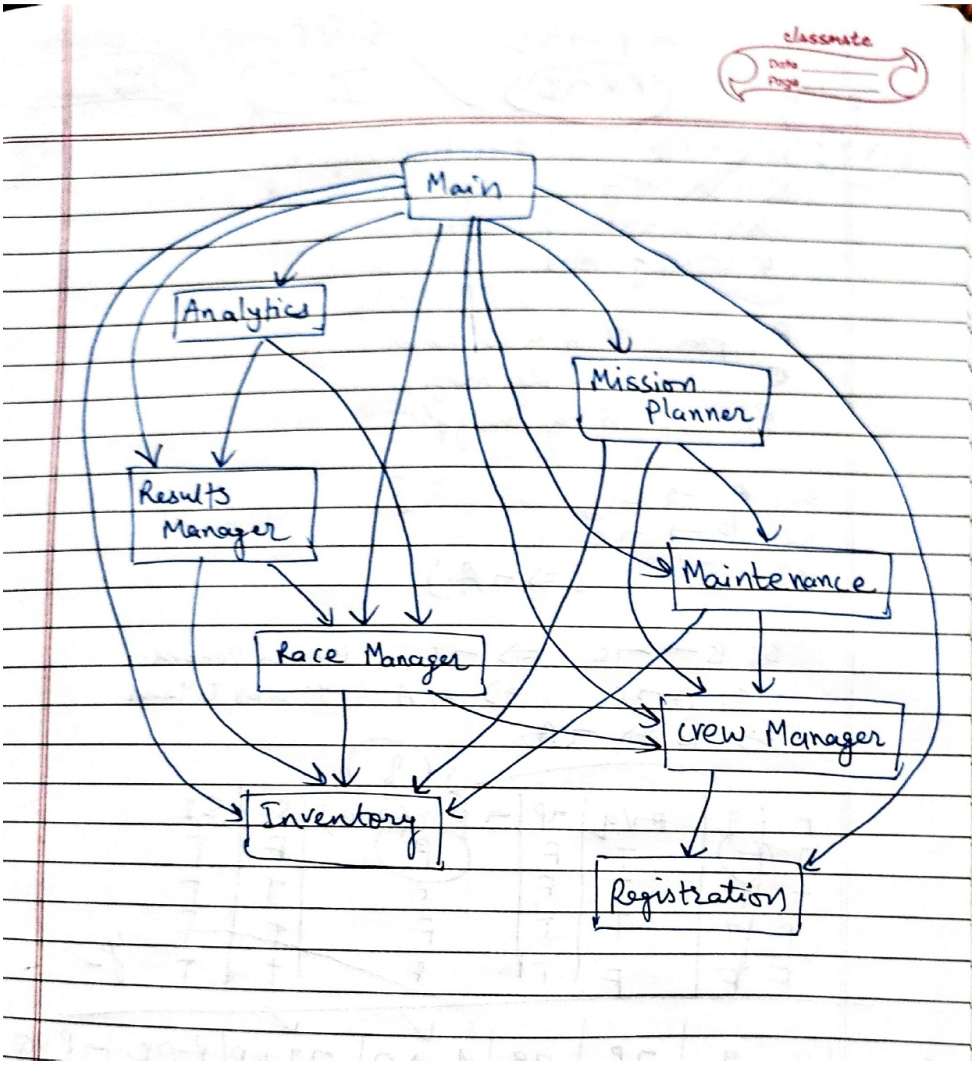
1 Extra Modules Implemented:

1. **Maintenance:** Handles repairs for damaged vehicles by verifying that the requested crew member holds the "mechanic" role.
2. **Analytics:** Statistics aggregator tracking individual driver performance (win counts) and lifetime prize money by sourcing data from Results and Race Management.

2 Compulsory Modules Implemented:

1. **Registration:** Core module handling unique ID generation and name registry.
2. **Inventory:** Core module managing monetary balance and vehicle status.
3. **Crew Management:** Assigns operational roles (driver, mechanic, strategist), dependent on Registration.
4. **Race Management:** Handles race creation and entry eligibility (requires a registered "driver" and a "good" condition vehicle), deducting entry fees via Inventory.
5. **Results:** Processes race outcomes, validates entries via Race Management, awards cash prizes to Inventory, and degrades vehicle status over time.
6. **Mission Planning:** Complex, high-level task runner. Requires specific roles and intact vehicles. Seamlessly coordinates with Maintenance if an assigned vehicle requires repairs before mission deployment.
7. **Main:** High-level composite module acting as the dependency-injected orchestration wrapper.

3 Call Graph (2.1)



4 Integration Tests Explanation:

1) `test_full_happy_path`

Why needed: Verifies end-to-end data flow across Registration, Crew Management, Inventory, Race Management, Results, and Analytics. Without this, we cannot confirm that modules cooperate correctly in the normal successful scenario.

2) `test_attempt_race_without_registration`

Why needed: Enforces the business rule that unregistered members cannot get roles. This protects downstream modules (race/mission) from invalid identities.

3) `test_mission_with_damaged_car_needs_mechanic`

Why needed: Validates Mission Planning + Maintenance + Inventory interaction. It confirms that a damaged car blocks missions unless a mechanic is available and repair succeeds.

4) `test_registered_non_driver_cannot_enter_race`

Why needed: A registered member is not automatically race-eligible. This test ensures Race Management depends on Crew Management role validation (driver-only policy).

5) `test_damaged_car_cannot_enter_race`

Why needed: Confirms Race Management reads vehicle condition from Inventory and blocks unsafe entries. Prevents race state from accepting invalid car status.

6) `test_race_entry_fails_if_inventory_cash_insufficient`

Why needed: Checks Race Management + Inventory fee deduction coupling. If cash is insufficient, race entry must fail and not add partial/invalid entries.

7) `test_results_reject_winner_not_in_entries_and_do_not_pay_prize`

Why needed: Validates Results integrity rules. A winner must be one of the actual entrants; otherwise no prize transfer and no history update should occur.

8) `test_mission_fails_with_wrong_mechanic_role_for_damaged_car`

Why needed: Ensures Mission Planning cannot bypass Maintenance role constraints. A helper with any role other than mechanic must not be accepted for repair flow.

9) `test_mission_fails_when_repair_needed_but_cash_too_low`

Why needed: Checks a critical edge interaction: Mission Planning triggers Maintenance, which depends on Inventory cash. When repair cannot be paid, mission must fail and car state must remain damaged.

10) `test_analytics_aggregates_multiple_results_correctly`

Why needed: Verifies Analytics consistency with Results and Race Management across more than one race. Ensures per-driver win counts and total prize aggregation are both correct.

Detected errors and fixes summary

- **Detected:** No failing functional defect in the current implementation from these integration tests.
- **Validation:** Entire integration test suite passes on running the test cases.